TASK 29 • DATA SCIENCE TRACK

Seaborn Fundamentals

Prepared for: Abhishek Kumar

Objective: Learn how Seaborn simplifies statistical data visualization, select appropriate chart types, interpret patterns responsibly, and implement a reusable visualization workflow.

Included: count plot, box plot, histogram with KDE, scatter plot, correlation heatmap, pair plot, comparison with Matplotlib, efficient programming practices, troubleshooting, and interview questions.

Dataset note: The bundled CSV files are illustrative, offline-friendly teaching samples. They are not the original Titanic or canonical Iris datasets. Their patterns demonstrate techniques, not official historical findings.

1. Learning Objectives and Workflow

After completing this task, you should be able to identify categorical versus numerical variables, map an analytical question to a suitable chart, use Seaborn's DataFrame-aware API, customize titles and labels with Matplotlib, read basic statistical summaries, save figures, and explain the limitations of a visualization.

Recommended workflow: (1) identify the question, (2) inspect rows, columns, types and missing values, (3) select variables, (4) choose a chart, (5) label and format it, (6) interpret visible patterns, (7) check whether the interpretation is supported by the data, and (8) save the output and document the source.

2. Setup and Project Structure

Use Python 3.10 or newer. Install packages with `python -m pip install -r requirements.txt`. Run `python seaborn_fundamentals.py` from the project folder. To use the notebook, run `jupyter notebook Task_29_Seaborn_Fundamentals.ipynb`.

Recommended project structure: `data/` stores source CSV files, `visualizations/` stores exported charts, the Python script is the repeatable entry point, the notebook supports interactive learning, and the README documents setup and results.

3. Data Inspection and Validation

```
import pandas as pd

passengers = pd.read_csv("data/titanic_sample.csv")
flowers = pd.read_csv("data/iris_sample.csv")

print(passengers.head())
print(passengers.info())
print(passengers.isna().sum())
print(flowers.describe(include="all"))
```

Inspect column names, data types, sample size, missing values, and duplicates before plotting. A chart can be technically correct but analytically misleading if a column has the wrong type, a unit is unclear, or many observations are missing. Validate required columns early so spelling errors and schema changes are caught before chart generation.

Passenger-style sample

Columns include PassengerId, Survived (0 = no, 1 = yes), Pclass (1, 2, 3), Sex, Age, Fare, and Embarked. These fields support categorical counts and numerical distributions.

Flower-style sample

Columns include sepal_length, sepal_width, petal_length, petal_width, and species. The sample contains three teaching groups: setosa, versicolor, and virginica.

4. Core Seaborn Concepts

DataFrame-aware plotting

Functions such as `sns.scatterplot(data=df, x='column_a', y='column_b')` can use column names directly. This is often more readable than manually extracting arrays.

Axes, figures and layout

Matplotlib's `fig, ax = plt.subplots()` creates a figure and axes. Passing `ax=ax` to Seaborn makes the chart part of that axes. The figure is the whole canvas; an axes is the region containing a plot. Use titles, labels, legends and `tight_layout()` to improve readability.

Hue, style, size and bins

`hue` encodes groups with colors, `style` changes marker types, and `size` changes marker size. For histograms, `bins` controls how numeric values are grouped. Use these encodings sparingly so a chart remains easy to read.

Theme and palette

`sns.set_theme(style='whitegrid', palette='deep')` applies consistent defaults. Consistency helps a multi-chart report feel coherent; a palette should also remain legible and not be the only way group identity is communicated.

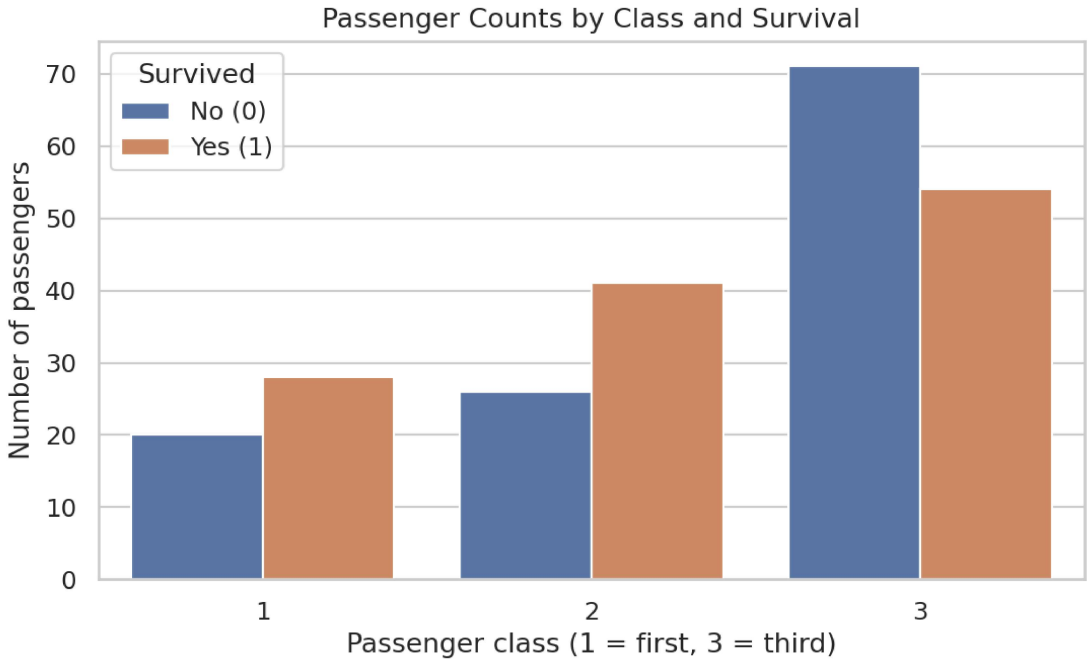
5. Six Visualizations with Code and Interpretation

5.1 Count plot — how many records are in each category?

A count plot counts observations in each categorical group. It is suitable for categories such as passenger class, species, product category, or response label.

```
fig, ax = plt.subplots(figsize=(8, 5))
sns.countplot(data=passengers, x="Pclass",
              hue="Survived", ax=ax)
ax.set(title="Passenger Counts by Class and Survival",
       xlabel="Passenger class", ylabel="Record count")
plt.tight_layout()
plt.show()
```

Interpretation: compare bar heights and subgroup counts. A count plot reports raw counts, not percentages. When category sizes differ, calculate proportions as well to avoid confusing a larger group with a higher rate.

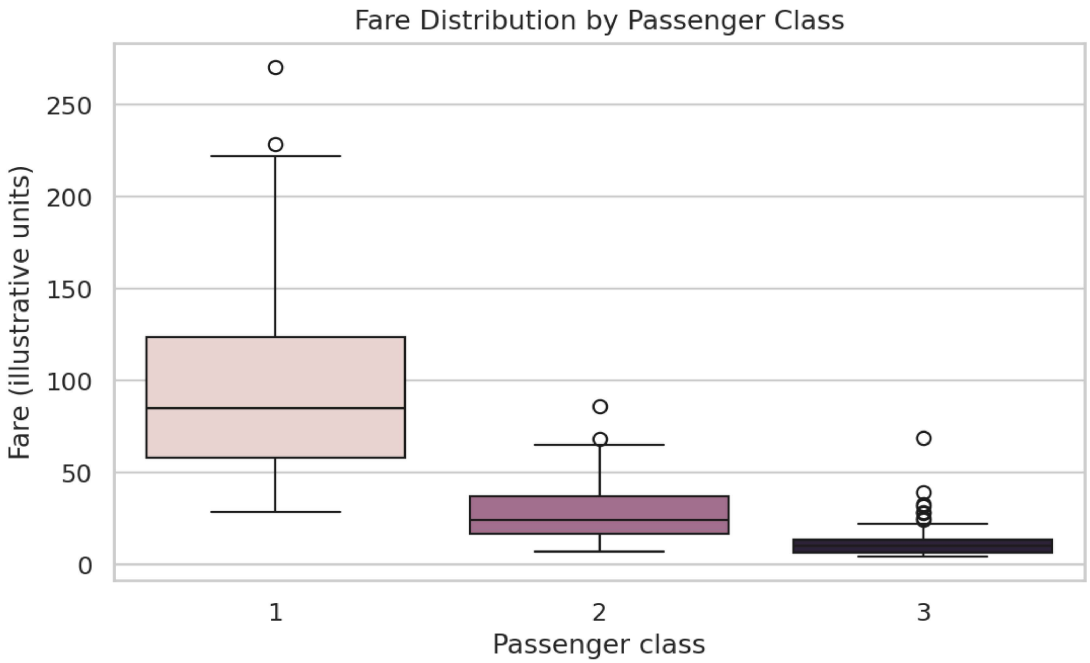


5.2 Box plot — how do distributions differ across groups?

A box plot summarizes a numeric variable by category. The median is the line inside the box; the box covers the 25th to 75th percentiles (the interquartile range, IQR). Whiskers follow the plotting rule, and points outside them can indicate potential outliers.

```
fig, ax = plt.subplots(figsize=(8, 5))
sns.boxplot(data=passengers, x="Pclass", y="Fare", ax=ax)
ax.set(title="Fare Distribution by Class",
       xlabel="Passenger class",
       ylabel="Fare (illustrative units)")
plt.tight_layout()
plt.show()
```

Interpretation: compare medians, IQRs and whisker lengths. A taller box indicates more spread in the middle half of the observations. An outlier is not automatically a data error; check context and source before removing it.

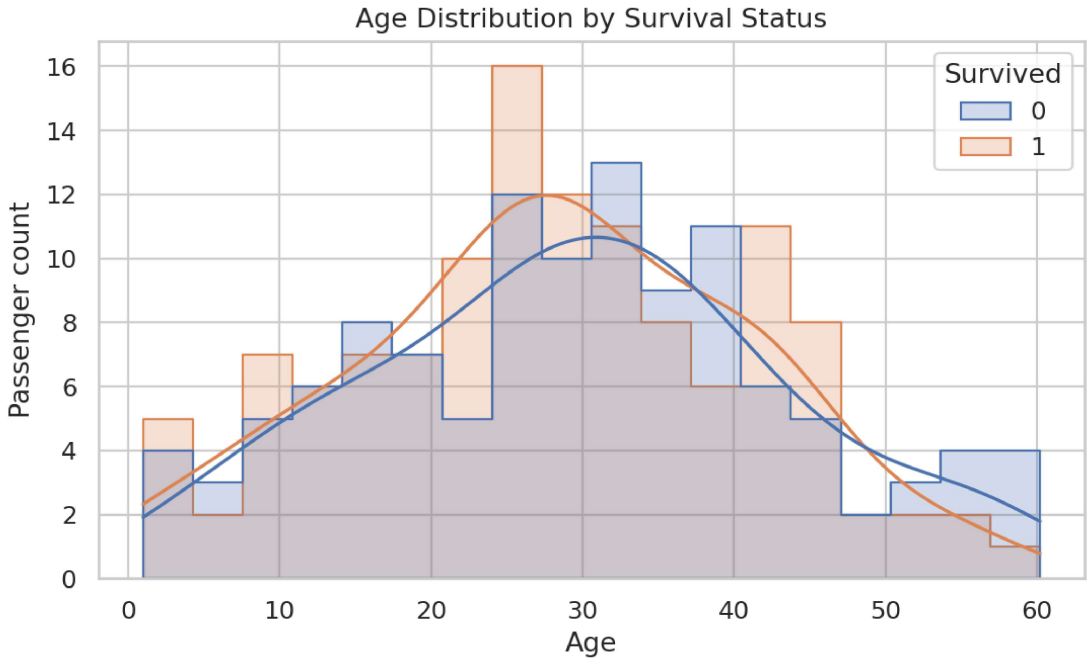


5.3 Histogram with KDE — what is the distribution shape?

A histogram divides a numeric variable into bins and displays the number of observations in each bin. KDE (kernel density estimation) draws a smooth estimate of distribution shape. KDE is a density estimate, not a count.

```
fig, ax = plt.subplots(figsize=(8, 5))
sns.histplot(data=passengers, x="Age", hue="Survived",
             bins=18, kde=True, element="step",
             common_norm=False, ax=ax)
ax.set(title="Age Distribution by Survival Status",
       xlabel="Age", ylabel="Count")
plt.tight_layout()
plt.show()
```

Interpretation: look for peaks, skew, gaps, and overlap between groups. Too few bins can hide structure; too many bins can exaggerate noise. KDE shape depends on smoothing bandwidth, so treat it as a guide rather than an exact representation.

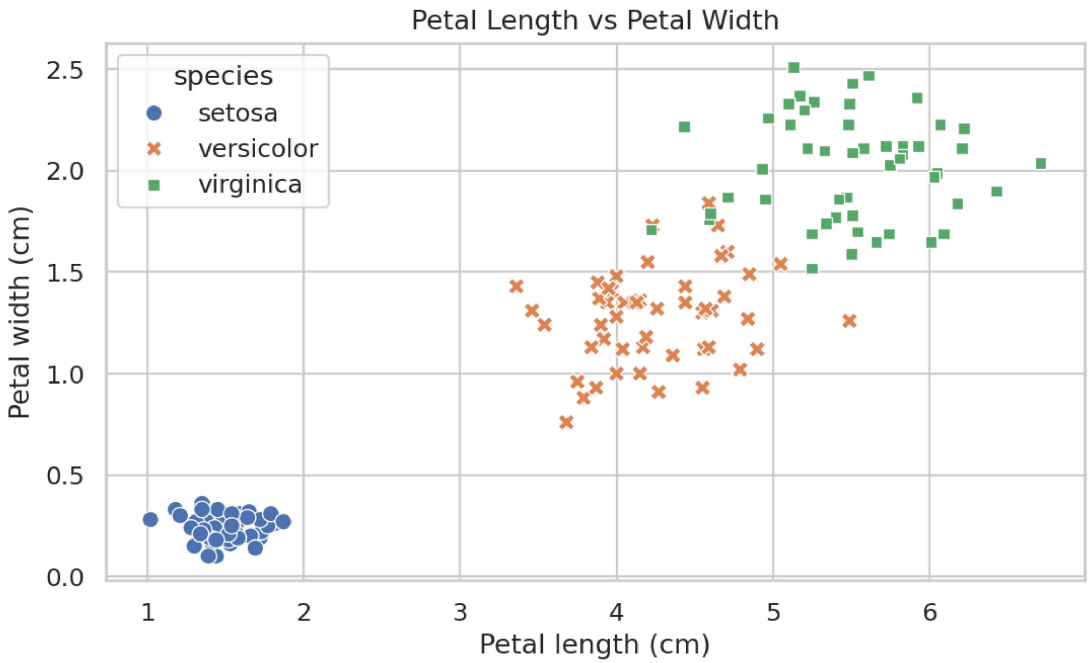


5.4 Scatter plot — how are two numerical variables related?

Each point represents an observation positioned by two numerical values. Hue and marker style can identify groups.

```
fig, ax = plt.subplots(figsize=(8, 5))
sns.scatterplot(data=flowers, x="petal_length",
                y="petal_width", hue="species",
                style="species", s=55, ax=ax)
ax.set(title="Petal Length vs Petal Width",
       xlabel="Petal length (cm)",
       ylabel="Petal width (cm)")
plt.tight_layout()
plt.show()
```

Interpretation: upward patterns suggest positive association; downward patterns suggest negative association. Clusters may indicate group separation. Dense overlapping points can hide records, so consider transparency or smaller markers when needed. Association alone does not prove causation.



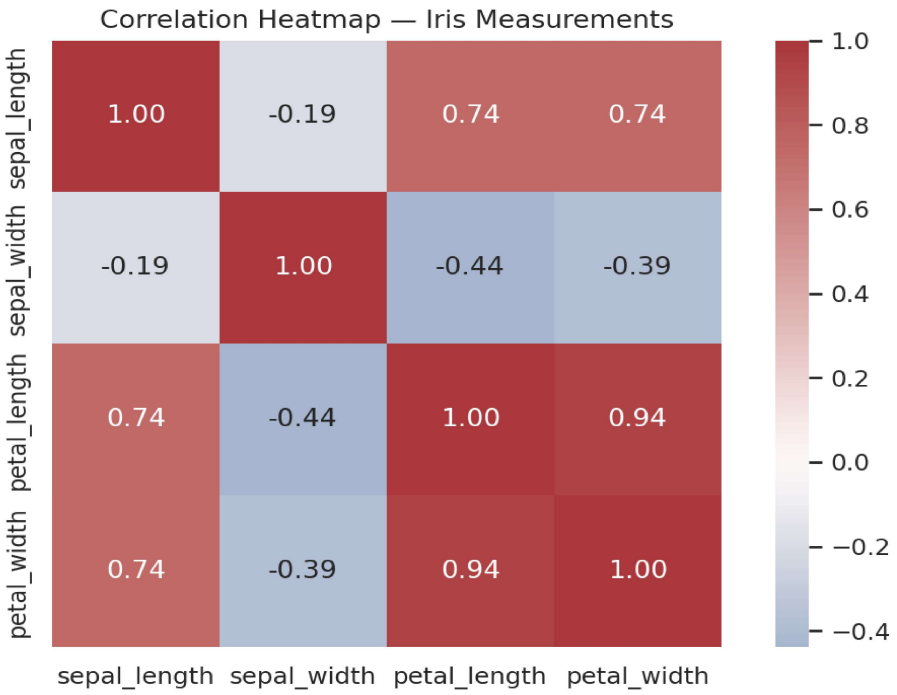
5.5 Heatmap — what relationships exist in a matrix?

A heatmap encodes values with color. A correlation matrix shows pairwise linear associations among numerical columns. Correlation values range from -1 to +1.

```
numeric = flowers.select_dtypes(include="number")
correlation = numeric.corr()

fig, ax = plt.subplots(figsize=(8, 6))
sns.heatmap(correlation, annot=True, fmt=".2f",
            cmap="vlag", center=0, square=True, ax=ax)
ax.set_title("Correlation Heatmap")
plt.tight_layout()
plt.show()
```

Interpretation: values near +1 mean strong positive linear association; values near -1 mean strong negative linear association; values near zero mean weak linear association. Correlation can miss nonlinear patterns and does not prove cause and effect.

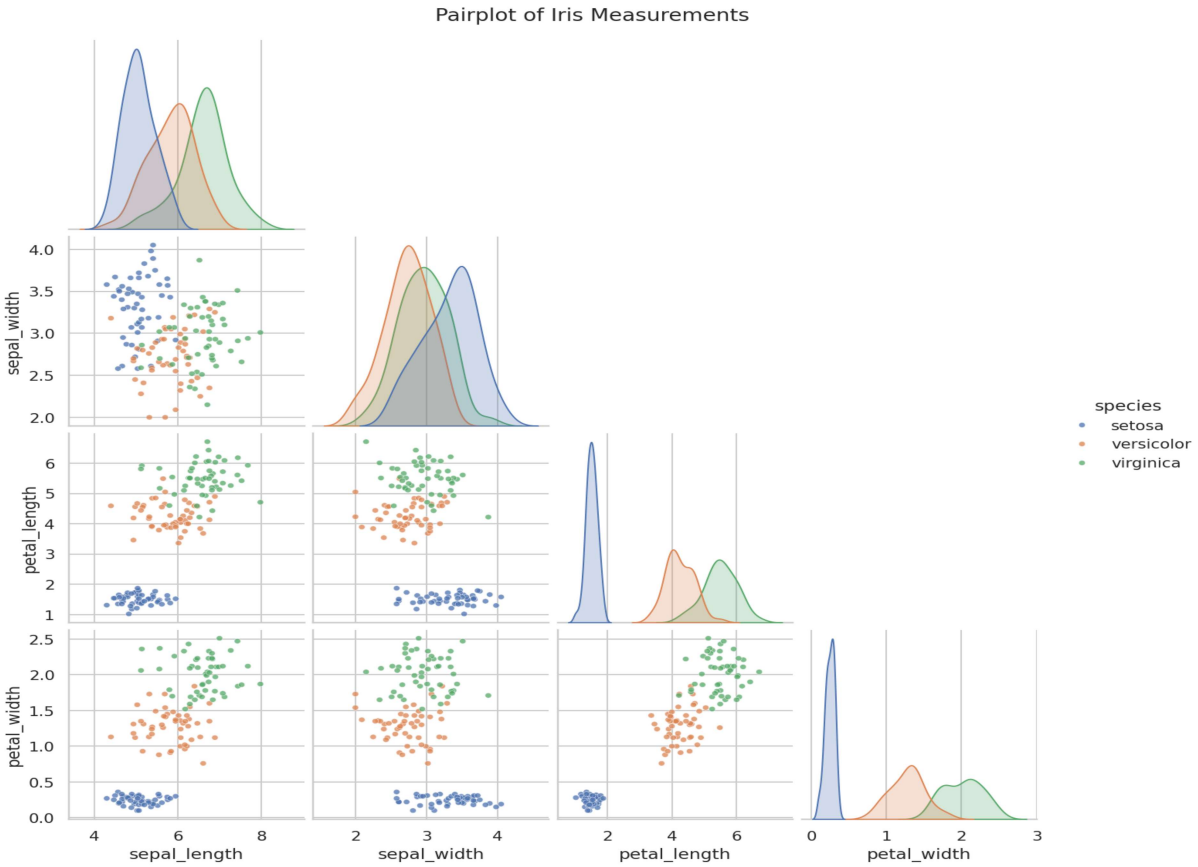


5.6 Pair plot — how do multiple features relate?

A pair plot combines one-variable distributions on the diagonal with scatter plots for pairs of variables off the diagonal. It is useful for broad exploratory data analysis, but large feature sets can create a crowded figure.

```
sns.pairplot(
    flowers,
    hue="species",
    vars=["sepal_length", "sepal_width",
          "petal_length", "petal_width"],
    corner=True,
    plot_kws={"s": 18, "alpha": 0.75}
)
plt.show()
```

Interpretation: scan for group clusters, correlated features, unusual points and potentially informative variables. Follow up with focused charts or statistical tests rather than drawing every conclusion from one pair plot.



6. Seaborn vs Matplotlib

Aspect	Seaborn	Matplotlib
Level	High-level statistical plotting	Flexible foundational plotting
DataFrame use	Convenient column-name interface	Supports arrays and DataFrames; often more manual
Statistical charts	Many ready-to-use distribution and group plots	Can build them, often with more code
Default appearance	Statistical themes and palettes	Very customizable; styling may require more setup
Control	Great for rapid exploration; accepts Matplotlib axes	Fine control over axes, artists, annotations and layouts
Best fit	Exploratory data analysis and comparisons	Highly customized or specialized figures

Seaborn is built on Matplotlib, not a competing replacement. A common workflow is to create a statistical chart with Seaborn, then use Matplotlib to adjust axes, labels, annotations, and layout.

7. Efficient and Maintainable Coding Practices

• **Load once:** read each CSV once, then pass the DataFrame to functions. Avoid repeated file reads inside plotting loops.

• **Separate responsibilities:** use one function per chart, a shared output directory, and a common save helper. This improves readability and makes individual charts easier to modify.

• **Validate early:** check that expected columns exist before chart creation. A clear validation error is easier to debug than a later KeyError.

• **Reuse configuration:** apply a single theme and consistent figure sizes, labels, and save resolution.

• **Close figures:** call `plt.close(fig)` after saving in batch jobs to release memory.

• **Compute once:** calculate the correlation matrix once and pass it to the heatmap instead of recomputing it.

• **Use meaningful names:** prefer `flowers`, `correlation`, and `output_dir` over cryptic single-letter names.

• **Keep charts honest:** preserve units, document transformations, check missingness, and avoid causal claims from correlations.

8. Common Errors and Fixes

• **ModuleNotFoundError:** install packages using the same Python interpreter that runs the script or Jupyter kernel.

• **FileNotFoundError:** run from the project root and verify the CSV paths. This project uses paths relative to the script location.

• **KeyError:** inspect `df.columns`; column names are case-sensitive and may contain spaces.

• **Unreadable chart:** increase figure size, add labels, use `tight_layout()`, and simplify legends.

• **Too many points:** reduce marker size, use transparency, or sample the data for an initial overview while keeping full data for final analysis.

• **Misleading group comparisons:** check whether groups have similar sizes and consider rates or normalized summaries in addition to raw counts.

9. Interview Questions and Answers

What is Seaborn?

A Python statistical visualization library built on Matplotlib with concise APIs and strong Pandas integration.

How is Seaborn related to Matplotlib?

Seaborn uses Matplotlib for rendering; Matplotlib can further customize Seaborn figures and axes.

When should you use a count plot?

When comparing the number of observations in discrete categories, optionally split by a second categorical variable.

When is a box plot useful?

When comparing medians, interquartile ranges, spread and possible outliers across groups.

What is the difference between a histogram and count plot?

A histogram bins a numerical variable; a count plot counts distinct categorical observations.

What does hue do?

It visually separates records by a categorical variable, usually using different colors.

What is KDE?

Kernel density estimation is a smoothed estimate of a distribution's density.

What does a correlation of -0.8 suggest?

A strong negative linear association in the observed data, not proof of causation.

What is a pair plot used for?

Exploring univariate distributions and pairwise relationships among several numerical features.

Are box-plot outliers always errors?

No. They are unusual observations under the plotting rule and should be checked against context and data quality.

Why validate columns before plotting?

It catches schema or spelling problems early and makes errors clearer.

Why close saved figures?

To release memory, especially when generating many plots in a script.